Ripple

系统分析与设计报告

围绕 CPU 开销的源码定位、对象建模与初步方案

摘要　Ripple 是基于 Java、Netty 与本地存储构建的配置 / 键值数据发布订阅系统。本报告从一次数据更新的处理流程出发，核对存储、确认追踪、订阅分发与后台任务的实现，发现 ACK 状态反复全量重建、数据操作附带清理、客户端临时线程池等潜在成本。报告选择 ACK 追踪作为首个研究对象，给出职责划分、渐进改造和验证设计。

项目边界

业务需求与参与者

参与者	需要系统完成的工作
配置维护人员	写入、删除或增量修改某个应用的一项数据。
订阅应用实例	按应用名称与键订阅，接收变更并维护本地数据。
部署与运维人员	配置节点及传播策略，观察运行状态并应对节点暂时不可达。

功能与质量要求

- F1：保留 Get、Put、Delete、Subscribe、Unsubscribe 及当前增量更新的可观察行为。
- F2：更新在目标服务器及订阅客户端间传播，接收端按消息标识处理重复。
- F3：保留确认进度、重试和重启后的恢复依据。
- Q1：在相同有效工作量下，降低 server 及参与 client 的 CPU 时间。
- Q2：维持可持续吞吐、传播延迟与数据正确性；记录队列和错误情况。
- Q3：把状态变化、计划构建与消息执行分开，使对象职责和生命周期清晰。

约束与判断标准

默认实现使用 SQLite、LastWriteWinsResolver 和 EqualDivisionClientDispatcher。根 POM 聚合 common、server、client 三个模块，server 与 client 依赖 common；源码目标字节码为 Java 8，采集 shell 的 JDK 为 27，后续对比必须固定实际运行版本。

> **优化问题的表述**
>
> 在功能、可靠性和负载条件可比的前提下，每完成一次有效更新，系统消耗多少 CPU？主要重复工作在哪里？通过怎样的职责调整可以减少它？

数据管理与传播策略

Server

Main → RippleServer → Node
组织存储、订阅、ACK 追踪、健康状态和传播策略

Client

Main → RippleClient
选择服务器，发起操作，接收 SYNC / DISPATCH，维护本地存储

↕ 自定义消息 / Netty / TCP ↕

Common

Item、节点元数据、更新消息、编解码、存储接口与默认合并规则

现有核心对象及责任

对象 / 接口	职责与变化原因
Node	协调写入、订阅通知、确认初始化和服务器间传播。
Storage / 各 Service	保存数据项、更新消息和 ACK；默认实现为 SQLite。
Overlay	根据来源、上游和当前节点计算后续同步目标。
ClientDispatcher	根据订阅列表选择直接推送或委托客户端转发。
Tracker / Worker	记录确认状态、选择重试目标；周期执行维护任务。
NodeSelector	客户端根据键的哈希候选及可选的 CPU 负载选择服务器。

已有扩展点与性能含义

Overlay、ClientDispatcher、NodeSelector、Storage 都提供策略或接口边界。它们允许改变传播、分发、路由及存储，但不同策略承担的工作并不相同。server CPU 降低可能伴随 client CPU 上升，改变副本目标数量会改变语义与工作量。

以一次更新为例

序	对象	执行内容
1	RippleClient / Api	选择服务器、复用或建立连接，编码并发送 PutRequest。
2	MessageHandler	按类型找到 PutRequestHandler，直接调用业务处理。
3	Node	创建 UpdateMessage，查重、查 item、保存消息。
4	Tracker	源服务器初始化 ACK 目标和进度。
5	ClientDispatcher	找订阅者，发送 SYNC 或 DISPATCH。
6	Node 执行器	异步计算并执行服务器间同步。
7	接收客户端	保存更新，DISPATCH 接收者还承担向其他客户端转发。

连接层与业务层的边界

按当前示例配置，client 主动连接 server 的 TCP 3001 端口，订阅请求携带 client 的地址和监听端口 5001，server 建立回连用于推送。HTTP 4001 / 6001 服务于网页操作。TCP 提供字节流，消息类型、UUID、业务响应和确认追踪由 Ripple 定义。连接可复用，每条更新不重新握手。

两个并行观察入口

前台：每条更新

SQL、ACK 初始化、订阅分发先执行，服务器同步随后提交异步任务。

后台：Worker

重连、健康检查、Tracker.retry 和 CPU 采集，每轮末尾休眠 10 秒，周期还包含执行耗时。

> **源节点返回成功的边界**
>
> 主流程不会等待所有服务器和订阅客户端完成更新。因此，实验中的完成数应该定义为目标接收端确认完成。

可能的优化方向和待验证假设

优化方向

方向	代码现象	触发条件
ACK 追踪	反复全量重建状态	ACK 表记录多；单节点也执行
存储清理	普通操作附带多条 SQL	数据项多、操作密集
客户端转发	每消息创建线程池	同一键至少 4 个订阅 client
分发计划	重复排序、分组、签名	订阅稳定且更新频繁
日志 / 编解码	逐消息格式化与对象分配	高消息率或大 value

研究假设

- H1：ACK 记录增加时，全量状态重建会放大源写入与后台重试的 CPU 工作。
- H2：在多数据项场景，索引缺失与高频清理会显著增加数据库执行成本。
- H3：多订阅者场景中，复用执行器和分发计划可减少线程及对象创建。

getter 隐藏了全量数据库重建

```
    private Map<UUID, AbstractMessage> getPendingMessages() {
        Map<UUID, AbstractMessage> map = new ConcurrentHashMap<>();
        List<Ack> ackList = this.getNode().getStorage().getAckService().getAllAcks();
        for (Ack ack : ackList) {
            AbstractMessage message =
this.getNode().getStorage().getMessageService().getMessageByUuid(ack.getMessageUuid());
            map.put(message.getUuid(), message);
        }
        return map;
    }

    public void retry() {
        Set<UUID> keys = this.getPendingMessages().keySet();
        for (UUID uuid : keys) {
            this.retrySending(uuid);
        }
    }
```

每次调用都会查询全部 ACK，逐条查询消息并创建新 Map。该方法没有返回长期缓存，也没有筛选真正未完成的任务。

调用结构放大成本

设 ACK 表有 P 条记录，循环期间记录稳定、查询成功。一次重建执行 $P+1$ 条 SELECT，retry 先重建一次，逐条 retrySending 内部又各重建两次，另查单条 ACK。

$$N(P) = (1+2P)(P+1) + P = 2P^2 + 4P + 1$$

生命周期错配

`initProgress` 中的 `getPendingMessages().put(...)` 先重建旧状态，再向临时 Map 写入新消息并丢弃。真正持久化由后续 initAck 完成。已完成 ACK 未立即移出全量读取范围，因此该问题不只出现在故障期间。

> **需求与实现的矛盾**
>
> 需求是找到仍需重试的任务，实现却不断重建全部历史相关状态。首个方案应减少状态读取的范围和次数。

重复回收历史

```
applyMessageToStorage(message)
  → getItem(application, key)
      → recycle(application, key)
          SELECT before UUIDs
          SELECT latest boundary ORDER BY time LIMIT 3
          DELETE older messages
          SELECT after UUIDs → compare sets → clean ACK
      → SELECT item
  → newMessage(message) → duplicate check → INSERT
```

查重在传播入口和插入服务内部重复进行。getItem 的清理会出现在写入路径，client 收到 SYNC 时也会发生。TCP Get 请求还可能通过 getItem 和 findMessages 两次触发清理。

数据规模与索引

message 表以 UUID 为主键，源码没有创建匹配 `(item_application_name, item_key)` 筛选的索引。默认目标是每个数据项保留 3 条历史。清理先于插入，时间戳相同的历史可能超出目标数。总数据项增加仍会扩大表规模。

初步方案与必要约束

方案	边界与验证要求
候选复合索引	应用名、键、时间降序，先检查现有数据库索引与查询计划，评估额外写成本。
清理职责分离	按阈值 / 批次触发，不能删除尚需重试的消息或增量更新依赖的基准消息。
简化集合比较	确认数据库连接有效，能自动删除后避免重复删除操作
存储原子操作	统一去重与写入、事务归属和资源关闭，区分重复与真实写失败。

SQLite 扫描、排序和 JSON 构建会消耗 CPU，提交磁盘等待、锁等待会拉长延迟。优化前应区分这两类现象。

区分资源生命周期与消息生命周期

临时线程池

每条 DISPATCH 命中列表缓存后：
newFixedThreadPool(F)
→ 提交 F 个任务 → shutdown
高频时反复创建原生线程。

重复分发计算

每次更新：排序全部订阅者
→ 分组 → 再排序
→ 拼接列表 → 摘要及十六进制转换。

验证条件

默认分 3 组。同一数据项有 1～3 个订阅者时，每组至多一人，服务器直接 SYNC；从 4 个订阅者开始，才会出现组内代表接收 DISPATCH 再转发。验证条件应按同一数据项的订阅者数设置。

输入变化频率与计算频率的矛盾

订阅成员通常比数据值变化得慢，但现有实现每条更新都重算分发计划。设更新频率为 U、成员变化频率为 V，当 U 远大于 V 时，可把主要计划构建工作从随 U 执行调整为随 V 执行。

候选方案

- Client 持有长期、有界的转发执行资源，启动时创建、停止时关闭，列表响应与缓存命中路径统一复用。
- 为数据项维护不可变 DispatchPlan，包含成员版本、分组、代表和签名；成员或有效性变化使计划失效。
- 第一小步可用 StringBuilder 保持原签名输入格式，避免逐轮拼接不可变字符串；不改变摘要或线协议。
- 对同目的地维持次序，限制积压，处理冷连接、缓存淘汰和并发变更。

> **性能目标必须覆盖两端**
>
> 委托客户端转发可能减轻 server，却增加所有 client 合计 CPU 占用。需要同时报告单节点瓶颈和全系统 CPU 占用。

明确状态、策略与执行

计划内容

对象	主要责任	生命周期 / 所有者
PendingDelivery	消息标识、目标集合、已确认集合、下次重试时间。	一次传播任务
PendingDeliveryRepository	原子记录进度，批量读取未完成任务，支持恢复。	Node / Storage
RetryScheduler	按到期时间调度，使用单轮快照，退避并限制工作量。	Node
SubscriptionRegistry	维护数据项的成员快照和版本。	Node
DispatchPlan	不可变的分组、代表、签名与成员版本。	某数据项某版本
ClientForwarder	连接复用、异步发送、有界排队、错误与顺序处理。	RippleClient

对象协作

确认追踪路径

Node → Repository：创建进度
Scheduler → Repository：取到期快照
Scheduler → 发送接口：执行
ACK 处理 → Repository：原子更新

订阅分发路径

订阅变更 → Registry：递增版本
数据更新 → 取得匹配版本 Plan
Dispatcher → 执行计划
ClientForwarder → 发给组内成员

附录

大模型使用

模型辅助部分

根据作业要求利用大模型进行了代码结构、核心对象与执行流程的理解，问题分析和方案初稿。

手动核查部分

确认了模型所述的结构、对象、流程、问题在代码中确实存在，确认了初步方案的合理性，修改了缺乏上下文情况下不明确的表述。